Towards Repository-Level C++ to Java Translation with Large Language Models

Anonymous Author(s)*

ABSTRACT

Code translation—converting code from one programming language to another while preserving functionality—is critical for legacy system migration, cross-platform development, and ecosystem modernization. Traditional rule-based or manual approaches are limited in scope and efficiency. While large language models (LLMs) have shown promise in code generation, most existing research focuses on translating isolated methods or small files, leaving project-level translation—where accumulated language paradigm differences become significant—largely unexplored.

This paper proposes TOOL, a hierarchical, macro-to-micro translation framework designed to mitigate error propagation in large-scale code translation. The framework first performs static analysis to extract call graphs and decompose code into functional modules using a custom Java parser tool. Translation then proceeds in two layers: (1) at the class level, where overall architecture and header files are designed with language-specific pattern mapping, and (2) at the method level, where implementations are translated bottom-up along call chains to ensure dependency stability. To bridge library and runtime differences, the framework dynamically generates necessary dependencies and an autonomous agent iteratively compiles, tests, and repairs the translated code using intelligent error analysis and fix strategies.

We evaluate TOOL by translating several cohesive Java modules to C++, two languages with significant differences in underlying execution models. Experimental results demonstrate that ... TODO: Add experimental results

Keywords: Code translation, Large language models, Project-level translation, Java, C++, Static analysis, Hierarchical translation

1 INTRODUCTION

The increasing need for software migration, cross-language interoperability, and multi-platform deployment has made automated code translation a valuable research area. Early techniques relied on handcrafted rules or template-based transformations, which are difficult to scale. Recent advances in LLMs have enabled more flexible and context-aware code generation, yet their application in translation remains largely confined to small, self-contained code snippets. Translating entire projects or interconnected modules introduces challenges such as architectural mismatch, dependency management, and compounding semantic gaps between languages—especially those with different runtime paradigms like Java (managed, JVM-based) and C++ (unmanaged, native).

Our work is situated at this intersection. We aim to develop a translation framework that leverages the generative strength of LLMs while deeply integrating extensible, domain-aware knowledge and validation mechanisms. We target translation tasks involving complex algorithm logic and API migrations (akin to Type 3/4 in the taxonomy by Jiao et al.[7]). Inspired by hybrid systems like IRENE [10] and compositional approaches like AlphaTrans [5], we propose a more generalized and extensible translation knowledge orchestration method. This approach is designed to dynamically retrieve and apply relevant constraints and patterns, ensuring high semantic fidelity and project-level usability without sacrificing flexibility.

This paper addresses the problem of error accumulation in project-level translation. We argue that translating files or methods independently without a high-level understanding of the module's purpose leads to irreconcilable inconsistencies. Instead, we propose a top-down understanding followed by bottom-up translation, where the LLM first comprehends the macro-structure of a functional module, plans the overall translation strategy, and then executes translation in a layered manner.

Our main contributions are:

- **A hierarchical translation framework (tool)** that separates class-level design from method-level implementation, enabling macro-to-micro translation with reduced error propagation.
- **A call-graph-guided, bottom-up translation order** that stabilizes dependencies by translating leaf methods first, ensuring callees are available when translating callers.
- **An autonomous compilation and repair agent** that validates and corrects the translated code using intelligent error analysis, supporting iterative compilation with automated fix strategies.

2 RELATED WORK

2.1 Large Language Model Hallucination

Rawte et al. [16] categorize hallucinations in large foundation models into four modalities: text, image, video, and audio, and propose evaluation criteria for assessing their severity. In this work, we focus specifically on text hallucinations generated by large language models. Hallucination constitutes a security-relevant risk in large language models, as Yao et al. [20] demonstrate that adversarial-like nonsensical inputs can provoke hallucinated outputs. To effectively detect hallucinations, Chen et al. [2] propose a robust discriminator named RelD to effectively detect hallucination in LLMs' generated answers based on a bilingual question-answering dialogue dataset RelQA. Manakul et al.[12] proposes SelfCheckGPT, which leveraging the idea that if an LLM has knowledge of a given concept, sampled responses are likely to be similar

and contain consistent facts. To effectively mitigate hallucination, Ji et al. proposes to mitigate LLM hallucination via self-reflection [6], while Roychowdhury et al. [17] present the three major stages in designing hallucination-minimized LLM-based solutions in the financial domain.

Given that code is a safety and functionality-critical output of LLMs, researchers have increasingly focused on hallucinations in generated code. Liu et al. [8] conduct an empirical study of hallucinations in LLM-generated code and identify three primary types: requirement conflicts, knowledge hallucinations, and code inconsistencies. Their results reveal severe consequences, where a dominant proportion of hallucinations lead to functional errors, with hallucinated code achieving a low pass@1 rate. Extending this line of work to repository-level generation, Zhang et al. [23] further categorize hallucinations into task requirement conflicts, factual knowledge conflicts, and project context conflicts. They also analyze the causes of these hallucinations, attributing them to factors such as low-quality training data, limitations in intent comprehension, and insufficient context awareness. To tackle the hallucination in generated code, Zhang et al. [23] demonstrates the effectiveness of retrieval-augmented generation, improving pass@1 scores across all models. Other approaches include self-refinement prompts, chain-of-thought reasoning, and specialized code models. CITATION?

2.2 Code Translation

Due to the rich syntactic structure of code, early seq-to-seq approaches frequently generated syntactically invalid outputsCITATION. To address this, Chen et al. [1] use of tree-to-tree neural networks, which encode and decode abstract syntax trees (ASTs) to explicitly model program structure. Drissi et al. [3] further refine this approach by introducing a grammar-driven tree-to-tree model that hard-codes target language syntax rules into the decoder, guaranteeing grammatical correctness. However, these works were limited in generalization and scalability by their dependence on parsers, and manually defined grammars.

As the field evolves, data-driven and unsupervised methods take center stage, reducing reliance on handcrafted rules and expert knowledge. Roziere et al.[18] introduced TransCoder, which leverages back-translation and denoising auto-encoding on monolingual code corpora to learn cross-lingual code representations without supervision. Meanwhile, pre-trained models such as CodeBERTCITATION and CodeT5CITATION leverage large-scale code corpora to capture semantic and syntactic patterns via general-purpose representation learning, significantly improving code understanding and generation. However, Malyala et al.[11] found that those models also exhibit systematic errors in complex translation scenarios, such as type conversions and loop restructuring. Pan et al.[15] show that LLMs introduce a significant number of bugs during translation, highlighting the limitations of statistical approaches in understanding deep semantic and syntactic constraints.

Recently, research emphasizes augmenting LLMs with programming language-specific knowledge, such as type systems, safety rules, and API mappings. Luo et al. [10] proposed the IRENE framework, which combines static analysis to extract "high-risk patterns" with retrieval-augmented generation to guide the model toward producing Rust code that complies with ownership rules. Similarly, Hong et al. [4] focused on type migration, employing a call-graph-based strategy to coordinate interface changes across functions. These hybrid systems effectively address the "semantic drift" and safety violations common in naive LLM translations. Extending this neuro-symbolic approach, Ibrahimzada et al. [5] introduced AlphaTrans, which compositionally combines symbolic knowledge with neural models for repository-level translation and validation. Meanwhile, Nitin et al.[13] enhances translation by generating multi-modal specifications (e.g., comments, tests) to provide additional context. While these methods significantly improve reliability, they often require substantial engineering effort to construct and maintain domain-specific knowledge bases.

2.3 Repository Translation

As capabilities of the LLMs grow, the research scope has expanded from function-level snippets to more complex, practical scenarios. Repository-level code translation presents unique challenges, including managing inter-file dependencies, build configurations, and project-wide context. Wang et al. [19] introduced RepoTransBench, a large-scale, multi-language benchmark for evaluating repository-level translation, revealing that even state-of-the-art LLMs struggle with the overall success rates. To tackle this, Zhang et al. [22] proposed a skeleton-guided translation framework that first extracts a high-level structural skeleton in the target language before filling in implementation details. Yuan et al. [21] and Ou et al. [14] further explored project-level translation by synergistically integrating knowledge graphs with the LLM in the repository context. In parallel, Liu et al.[9] attempt to improve the human-in-the-loop experience. They developed hmCodeTrans, an interactive translation framework that allows developers to provide real-time feedback, significantly reducing human effort compared to traditional post-editing workflows.

3 EMPIRICAL STUDY

We conduct an empirical study to systematically examine the effectiveness and limitations of large language models for repository-level C++-to-Java translation.

Given that repositories may exceed model input limits, segmentation of the source code is required prior to translation. We consider two representative segmentation schemes that reflect common developer usage scenarios when applying large language models: *(a) File-by-file translation.* Each source file is translated independently without explicit cross-file contextual information. *(b) Method-by-method translation.* Each C++ method (function) is translated independently, allowing

finer-grained control while sacrificing broader cross-method context awareness.

- **RQ1**: How is the effectiveness of large language models in repository-level C++-to-Java translation using representative segmentation schemes?
- **RQ2**: What are the common failure patterns in repository-level C++-to-Java translation using large language models?

3.1 Setup

Dataset. We selected four real-world Java projects from open-source repositories to evaluate the translation performance. In selecting translation units, we chose a series of top-ranked projects from the Java section of GitHub Trending, while excluding very famous projects (to avoid relying on prior knowledge of large models) and projects that rely too much on language features (such as reflection) and ecosystems (such as Android projects). These projects span multiple domains to ensure diverse scenarios:

- **wso2-commons-httpclient**[1]: An HTTP client library (Cookie module), representing network communication utilities.
- **failsafe**[2]: A fault tolerance library (CircuitBreakerExecutor and RateLimiterExecutor modules), representing resilience patterns for distributed systems.
- **junit5**[3]: A testing framework (EnableJUnit4MigrationSupport module), representing testing infrastructure and migration utilities.

Given the large scale of the complete project, and for the sake of ease of experimentation, we extracted a fully functional module from the project as a translation unit. The specific method for generating translation units is as follows: First, randomly select one from the top-level functional files of the project. Then, search for all the code in the project that this file depends on, forming a complete and self-consistent unit. The example in 1 shows the dependencies translation unit for the Cookie module.

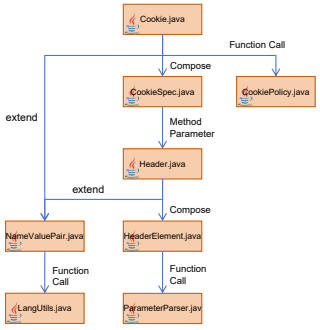

Figure 1: An example of a translation unit extracted from source project.

Subject Models. We evaluated three state-of-the-art large language models:

- **DeepSeek-V3.2**: An open-weight model with strong code generation capabilities.
- **Qwen-3.5Plus**: A model optimized for programming tasks.
- **ChatGPT-5.1**: A proprietary model with advanced reasoning abilities.

Evaluation Metric. We measure translation effectiveness using **compilation success rate**. We combine all the generated C++ source files and header files, and then compile each source file separately (the related header files will be compiled during this process). A successful compilation indicates that a .o file has been generated, the compilation success rate is defined as

$$\frac{FileSuccessfullyCompiled}{totalCppFileCount} \times 100\%$$

Procedure. For each model and segmentation scheme, we translated all Java files in the selected projects and attempted to compile the generated C++ code. We manually analyzed compilation failures and categorized errors into distinct types to identify systematic failure patterns. It is important to note that our manual analysis focuses on understanding the distribution and characteristics of identified problems, which reveal fundamental challenges in LLM-based code translation, rather than providing a complete enumeration of all possible defects.

3.2 RQ1: Translation Effectiveness

Figure 2 presents the overall compilation success rates across different models and segmentation strategies. We observe that all three models struggle with repository-level translation, with success rates ranging from 1.09% to 19.32%. This reveals a significant gap between the capabilities of current LLMs and the requirements of real-world project translation tasks.

Impact of Segmentation Strategy. Comparing the two segmentation schemes, file-by-file translation consistently outperforms method-by-method translation across all models. Specifically:

- **DeepSeek-V3.2**: 10.29% (file-by-file) vs 4.73% (method-by-method)
- **Qwen-3.5Plus**: 19.32% (file-by-file) vs 1.09% (method-by-method)
- **ChatGPT-5.1**: 7.43% (file-by-file) vs 6.04% (method-by-method)

The superior performance of file-by-file translation can be attributed to better preservation of contextual information. When translating entire files, models maintain awareness of class structures, member variable declarations, and inter-method relationships within the same compilation unit. In contrast, method-by-method translation isolates individual functions, forcing the model to infer dependencies without explicit context, leading to inconsistencies in type usage, API calls, and naming conventions.

Model Performance. Qwen-3.5Plus achieves the highest overall success rate at 19.32% under file-by-file translation, followed by DeepSeek-V3.2 (10.29%) and ChatGPT-5.1 (7.43%). However, Qwen-3.5Plus exhibits extreme variance between strategies (19.32% vs 1.09%), indicating high sensitivity to

[1] https://github.com/wso2/commons-httpclient
[2] https://github.com/failsafe-lib/failsafe
[3] https://github.com/junit-team/junit5

Table 1: Selected Java projects for empirical study.

Translate Unit	File Count	Source Project	Domain
Cookie	8	wso2-commons-httpclient	http client
EnableJUnit4MigrationSupport	32	junit5	unit test
CircuitBreakerExecutor	23	failsafe	fault tolerance
RateLimiterExecutor	24	failsafe	fault tolerance

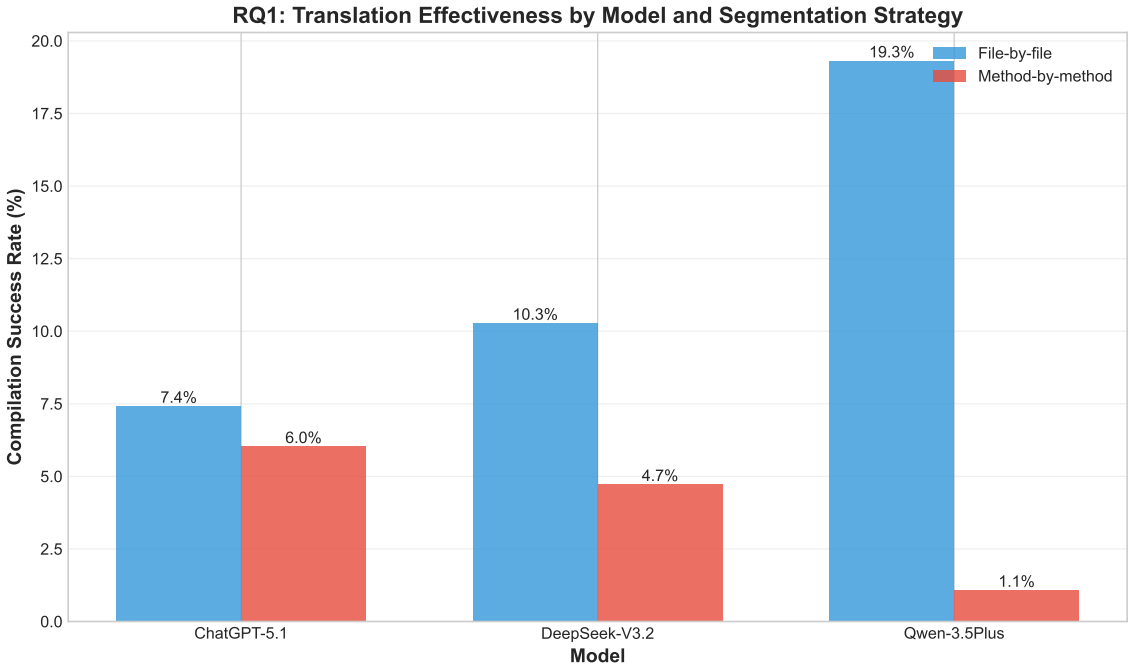

Figure 2: Overall compilation success rates by model and segmentation strategy. File-by-file translation generally outperforms method-by-method across all models.

segmentation granularity. This suggests that model architecture and training data significantly influence translation robustness across different input scopes.

Project-Level Analysis. Figure 3 reveals substantial variation in difficulty across translation units. The Cookie module from wso2-commons-httpclient achieves the highest average success rate (12.96%), while CircuitBreakerExecutor from failsafe proves most challenging (1.96%). The EnableJUnit4MigrationSupport module demonstrates relatively stable performance (13.04%) across most configurations. Several factors contribute to this variance:

- **Code complexity**: Modules with deeper inheritance hierarchies (e.g., CircuitBreakerExecutor) present more translation challenges due to virtual function dispatch and abstract base class dependencies.
- **External dependencies**: Projects heavily reliant on third-party libraries (e.g., HTTP client utilities) introduce uncertainty when mapping Java APIs to C++ equivalents.

Key Findings for RQ1.

(1) Current LLMs achieve low compilation success rates (1–20%) for repository-level translation, indicating substantial limitations in handling real-world project complexity.
(2) File-by-file translation outperforms method-by-method by 2–18×, highlighting the critical importance of contextual breadth for maintaining translation consistency, however, for larger files, the model's handling of the details of each method may be somewhat biased..
(3) Success rates vary significantly across projects (1.96–13.04%), suggesting that code complexity, domain patterns, and external dependencies are key factors influencing translation difficulty.
(4) Even the best-performing configuration (Qwen-3.5Plus, file-by-file) succeeds on fewer than 1 in 5 files, underscoring the need for improved translation methodologies.

3.3 RQ2: Failure Patterns

To understand the root causes of translation failures, we systematically categorized compilation errors and analyzed their distribution. Figure 4 presents the error type occurrence rates

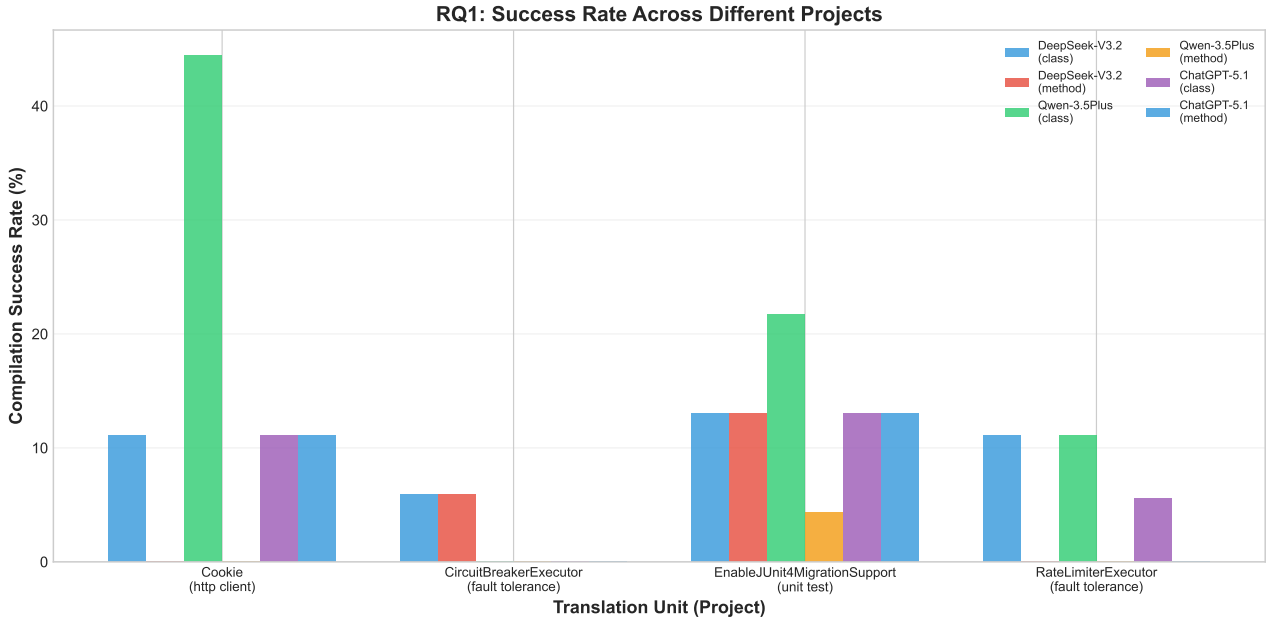

Figure 3: Compilation success rates across different projects. Performance varies significantly by translation unit, reflecting differences in code complexity and domain-specific patterns.

across all model-strategy combinations, revealing consistent patterns that transcend specific model architectures.

Error Type Taxonomy. We systematically categorized all compilation errors into 11 distinct types based on their underlying causes:

(1) **Build/Include Errors**: Missing header files, incorrect include directives, and dependency resolution failures.
(2) **Missing/Undefined Symbols**: References to undeclared functions, variables or classes.
(3) **Type System Errors**: Type mismatches, incorrect type conversions,and incomplete type definitions.
(4) **Syntax/Language Errors**: Violations of C++ syntax rules, improper use of language constructs, and semantic errors that prevent parsing.
(5) **Declaration/Definition Mismatch**: Inconsistencies between function signatures in headers and their implementations, or mismatched parameter lists.
(6) **Inheritance/Virtual Errors**: Issues related to virtual function overriding, abstract class implementations, and inheritance hierarchy problems.
(7) **Redefinition Errors**: Multiple definitions of the same variable, function, or class, often due to missing include guards or improper header structure.
(8) **Access Scope Errors**: Violations of access control rules (public/private/protected), incorrect friend declarations, and scope resolution failures.
(9) **Constructor/Destructor Errors**: Problems with constructor initialization, destructor implementation, and resource management lifecycle issues.
(10) **Template Errors**: Template declaration syntax errors, instantiation failures, and template parameter deduction problems.
(11) **Other Errors**: Compilation failures that do not fit into the above categories.

Dominant Error Categories. Across all configurations, three error categories account for the majority of methods failing compilation:

(1) **Build/Include Errors** This error category stems from three primary challenges:
- *Missing header files*: Sometimes the model does not include the corresponding header file when using classes from standard library.
- *Java-to-C++ mapping uncertainty*: For many classes in the Java standard library, such as Serializable and Comparator, the model cannot correctly map them to appropriate C++ classes.
- *Header file hallucination* Due to misleading import statements in Java code, models sometimes include header files that do not exist (for example, Java's Date class is often mapped to a file named 'Date.h', which does not exist in the C++ standard library).

(2) **Missing or Undefined Symbols** This category encompasses:
- *Untranslated dependencies*: When translating a method, models may reference methods from other classes that were translated with different signatures.
- *Fragmented type definitions*: Forward declarations may be omitted, or definitions may be split across headers incorrectly, causing redefinition errors.

(3) **Type System Errors**

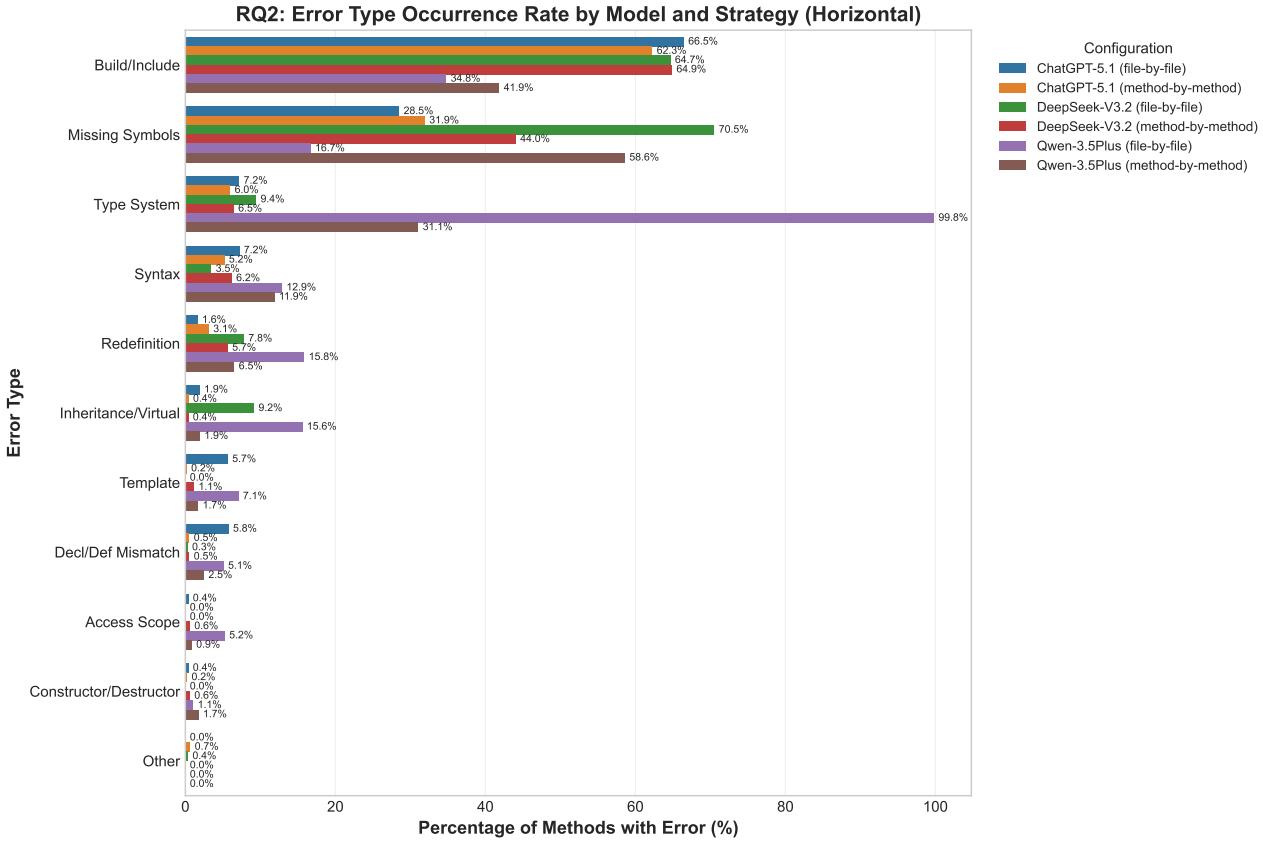

Figure 4: Error type occurrence rate by model and translation strategy. Each bar group shows the percentage of methods containing each error type, with different colors representing different model-strategy configurations. Build/Include and Missing Symbols errors dominate across all configurations.

- *Unsupported overload*: Some unsupported overloaded methods were used. For example, cannot stream std::exception_ptr directly with operator«.
- *Unsupported type conversion*: Some unsupported types were used. For example, cannot convert std::unique_ptr to std::string.
- *Wrong return type*: Incorrect return value types were returned.

Figure 5 illustrates the relative composition of error types across configurations.

Model-Specific Patterns. While overall error distributions are similar, subtle differences emerge:

- **Qwen-3.5Plus** exhibits higher Type System error rates (affecting over 99% methods in file-by-file strategy), suggesting weaker reasoning about generic type mappings and template constraints.
- **DeepSeek-V3.2** shows elevated Missing Symbols errors in file-by-file strategy (affecting over 70% methods), indicating difficulty maintaining API consistency across complex inheritance hierarchies.
- **ChatGPT-5.1** demonstrates relatively balanced error distribution but with consistently high Build/Include errors (affecting over 60% methods), reflecting challenges in header dependency resolution.

Strategy-Dependent Variations. Comparing segmentation strategies:

- **File-by-file** results in fewer methods with Missing Symbols errors on average, confirming that broader context helps maintain API consistency.
- **Method-by-method** generates more Redeclaration and Definition Mismatch errors, likely due to translating methods without visibility of existing declarations in headers.
- Both strategies exhibit comparable Build/Include error rates , indicating that header dependency resolution remains challenging regardless of segmentation granularity.

4 APPROACH

This section presents the detailed design of TOOL, our hierarchical framework for project-level Java to C++ code translation. We first provide an overview of the framework architecture, then describe each component in detail.

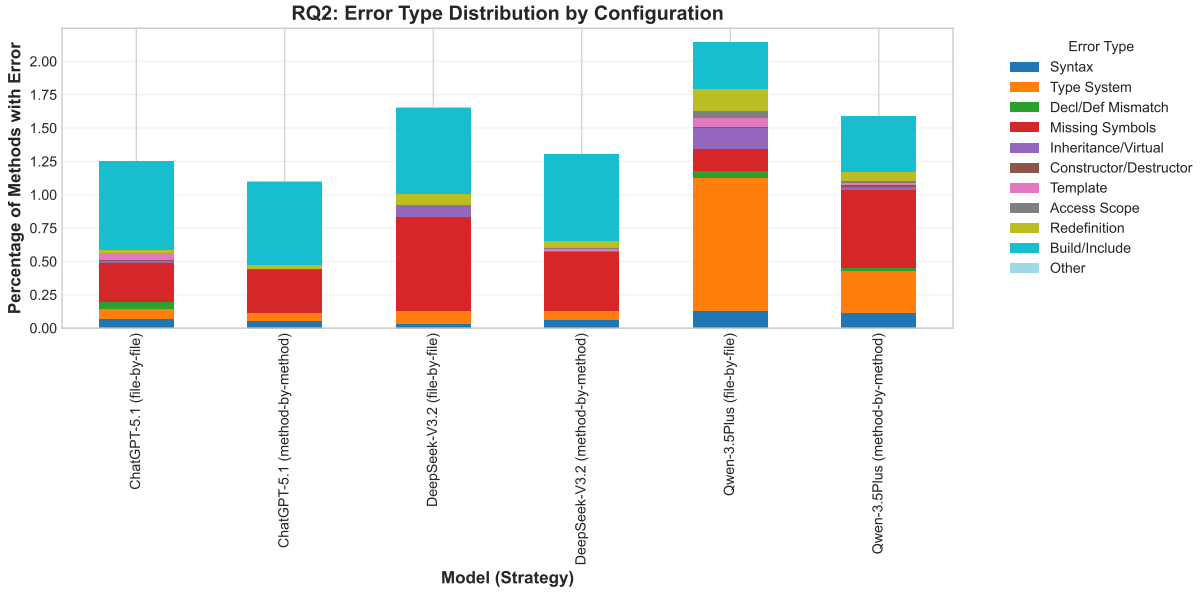

Figure 5: Stacked error distribution by configuration. Each bar shows the percentage of methods containing each error type. Build/Include and Missing Symbols dominate across all model-strategy combinations.

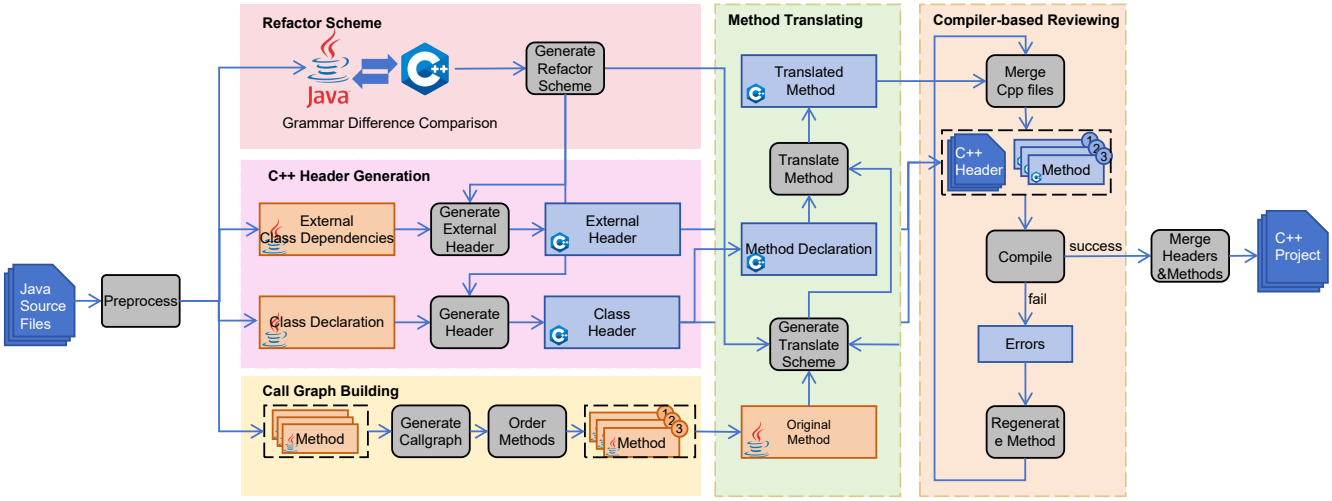

Figure 6: Overall architecture of tool showing the three-phase translation process.

4.1 Framework Overview

Figure 6 illustrates the overall architecture of TOOL. The framework adopts a **macro-to-micro translation strategy** that separates high-level structural design from low-level implementation details. This hierarchical decomposition addresses two key challenges in project-level code translation: (1) managing complexity at scale, and (2) mitigating error propagation through context-aware translation ordering.

The translation process consists of three main phases:

(1) **Static Analysis Phase**: A custom Java parser tool extracts the project structure, including class hierarchies, method signatures, and call graph dependencies. This phase decomposes the codebase into a hierarchical graph representation comprising *Header nodes* (classes, interfaces, enums) and *Method nodes* (individual method implementations). The graph captures both structural relationships (inheritance, type dependencies) and behavioral relationships (method call dependencies), which guide subsequent translation ordering.

(2) **Hierarchical Translation Phase**: The framework performs translation in two interdependent layers:

- *Header-level translation*: For each Java class, interface, and enum, the LLM generates a translation scheme that maps Java language constructs to appropriate C++ patterns (e.g., interfaces to abstract base classes with virtual functions, checked exceptions to error code handling). This layer establishes the structural foundation and type mapping conventions for the entire project.
- *Method-level translation*: Methods are translated following a bottom-up ordering derived from the call graph. Callee methods are translated before their callers, enabling that already-translated code serves as reference for translating dependent methods. This dependency-aware approach significantly reduces translation inconsistencies and API mismatches.

(3) **Validation and Repair Phase**: An autonomous compilation agent iteratively compiles the translated C++ code, analyzes compilation errors, and applies fixes using LLM-generated suggestions. The agent operates through a tool-use interface, invoking compilation and file modification primitives to iteratively resolve errors until compilation succeeds or maximum attempts are reached.

4.2 Static Analysis and Graph Construction

The first phase of TOOL involves parsing the Java source code and extracting structural information necessary for guiding the translation process. This phase constructs a project-level dependency graph that serves as the foundation for all subsequent translation decisions.

4.2.1 Project Graph Model. The static analysis phase constructs a bipartite graph representation consisting of two types of nodes:

- **Header Nodes**: Represent type declarations (classes, interfaces, enums). Each Header node captures:
  - The complete type declaration and field members
  - Inheritance relationships (parent class, implemented interfaces)
  - Import dependencies (external types required by this compilation unit)
  - A reference to all contained Method nodes
- **Method Nodes**: Represent individual method implementations. Each Method node captures:
  - The method signature (return type, parameter types, modifiers)
  - The complete method body implementation
  - Call dependencies (`children`: methods called by this method; `parents`: methods that call this method)
  - External call references (calls to methods outside the project boundary)

This bipartite structure enables separate handling of structural translation (Headers) and behavioral translation (Methods), while maintaining explicit relationships between them.

4.2.2 Call Graph Extraction. To establish the correct translation order for methods, the framework extracts the complete call graph from the Java source code. We integrate `java-callgraph2`, a static analysis tool that identifies method call relationships through points-to analysis.

The call graph construction process involves:

(1) Parsing method call traces from `java-callgraph2` output
(2) Matching caller and callee signatures to Method nodes in the project graph
(3) Establishing bidirectional edges between caller-callee pairs
(4) Identifying external dependencies (calls to library methods outside the project)

The resulting call graph enables topological sorting of methods into translation layers, where all dependencies of a method in layer n are guaranteed to be in layers 1 through $n-1$.

4.3 Hierarchical Translation Pipeline

The translation pipeline operates on the project graph in two distinct phases: Header-level translation establishes the structural foundation and type conventions, followed by Method-level translation that leverages dependency ordering for context propagation. Each phase employs a dual-pass strategy consisting of scheme generation followed by code generation.

4.3.1 Header-Level Translation. Header-level translation addresses the structural differences between Java and C++, establishing consistent conventions for the entire project before translating individual method implementations.

Scheme Generation. For each Header node, the LLM generates a **translation scheme**, a structured document that specifies:

- **Type mapping strategy**: How Java types map to C++ types (e.g., `ArrayList<T>` → `std::vector<T>`)
- **External dependency handling**: Identification of external dependencies that cannot be directly translated (e.g., third-party libraries, Java framework types), with strategies for mocking, replacing, or omitting them
- **Memory management strategy**: How to replace Java's garbage collection with appropriate C++ memory management (e.g., using stack allocation where possible, smart pointers for heap-allocated objects, RAII patterns for resource management)
- **Inheritance mapping**: How to map Java interfaces and abstract classes to C++ (e.g., interfaces → abstract base classes with pure virtual methods, default methods → virtual methods with implementations)
- **Generics conversion**: How to convert Java generics to C++ templates (including handling of bounded type parameters, wildcards, and type erasure implications)

The scheme serves as a contract between high-level design and detailed implementation, ensuring consistency across all methods within the class.

Header Translation. Using the generated scheme, the LLM translates each Header node to a C++ header file. The translation handles:

- Converting class declarations to C++ class syntax
- Translating field declarations with appropriate C++ types and initialization requirements
- Generating method declarations with translated signatures
- Adding necessary `#include` directives for both standard library and project dependencies
- Handling namespace declarations and forward declarations where appropriate

Importantly, in our framework, the translated C++ project structure may differ from the original Java structure while preserving core functionality. The dual-pass approach grants the LLM considerable flexibility during Header translation, particularly in scheme generation where structural refactoring decisions are made. For example:

- Java utility classes containing only static methods may be translated to C++ namespaces with free functions, eliminating the need for class instantiation
- Methods may be added, removed, or reorganized to better align with C++ idioms (e.g., separating declaration and definition, adding helper functions)
- Multiple Java classes in one file may be split into separate C++ header files, or vice versa, depending on compilation dependencies
- Interfaces may be merged or eliminated in favor of template-based designs where appropriate

This structural flexibility is particularly valuable when translating between languages with significant paradigmatic differences (e.g., object-oriented Java to multi-paradigm C++ with template metaprogramming), as it enables the translated code to follow idiomatic C++ patterns rather than mechanically mimicking Java's structure.

4.3.2 Method-Level Translation. Method-level translation focuses on translating implementation logic while maintaining behavioral equivalence. The key innovation is the bottom-up translation order, which enables effective context propagation.

Dependency-Aware Translation Ordering. Methods are grouped into layers using topological sort on the call graph:

- **Layer 1**: Leaf methods with no internal calls (pure computations, external library calls only)
- **Layer** n: Methods whose callees all reside in layers 1 through $n-1$

Methods within the same layer have no mutual dependencies and can be translated in parallel, providing opportunities for concurrent processing.

Scheme Generation. For each Method node, the LLM generates a detailed translation scheme that includes:

- **Functionality summary**: A concise description of what the method accomplishes
- **Implementation strategy**: How to achieve the same behavior in C++ (e.g., replacing Java streams with C++ algorithms, converting iterator-based loops with range-based for loops)
- **Context requirements**: Which already-translated methods to reference as examples
- **Exception handling**: How to convert Java checked exceptions to C++ error handling (e.g., return values, error codes, or C++ exceptions)

Because all callees are translated before their callers, the scheme can explicitly reference specific translation patterns from already-translated methods, promoting consistency across the codebase.

Method Translation. Methods are translated layer by layer. For each method, the LLM receives:

- The original Java method code
- The method's translation scheme
- **Translated code of all callees**: This provides concrete examples of how similar constructs were translated, enabling *pattern-based consistency*
- The translated Header for context on available types and APIs

This context-rich prompting significantly reduces hallucination by providing the LLM with concrete examples of how similar language constructs were translated elsewhere in the project.

4.4 Validation and Repair

The final phase addresses compilation errors through an autonomous agent that iteratively analyzes, diagnoses, and fixes translation defects. This phase bridges the gap between syntactically correct translation and semantically correct, compilable code.

4.4.1 Compilation Agent Architecture. The Compilation Agent operates as a closed-loop system with four core capabilities accessed through a tool-use interface:

(1) **compile**: Invoke the C++ compiler on a specific source file and capture the output
(2) **read_file**: Read the current contents of any source or header file
(3) **write_file**: Replace the contents of a file with modified code
(4) **create_file**: Generate new header or source files as needed

The agent maintains a conversation history with the LLM, providing compilation errors as feedback and requesting fixes. This interactive process continues until compilation succeeds or maximum attempts are reached.

4.4.2 Iterative Repair Process. For each C++ source file in dependency order, the agent executes the following loop:

(1) Invoke `compile` on the target source file
(2) If compilation succeeds, proceed to the next file
(3) Otherwise, provide the compilation error output to the LLM

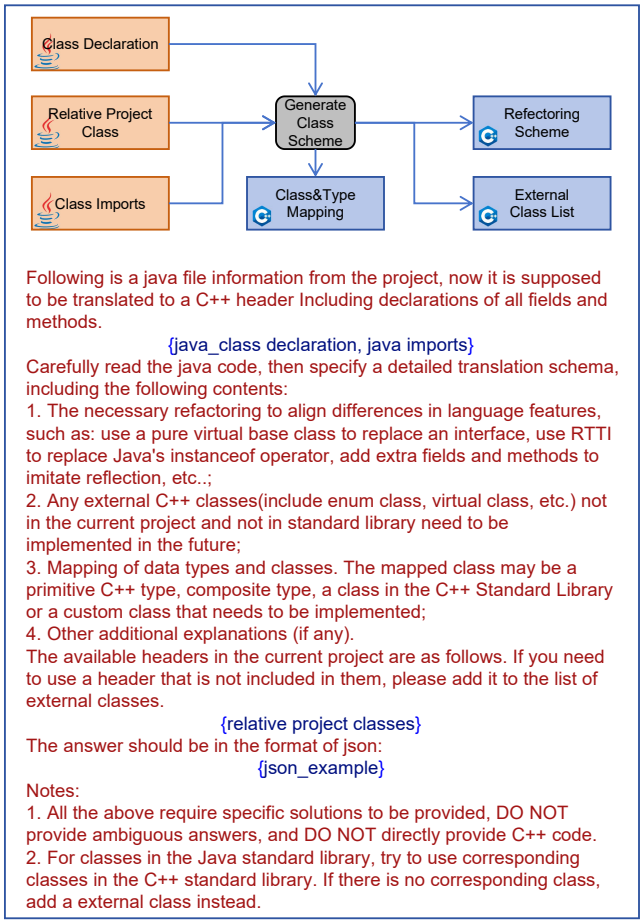

Figure 7: Header-level translation scheme and data flow. The LLM analyzes the Java class structure, generates a translation scheme specifying type mappings and structural changes, then produces the corresponding C++ header file.

(4) The LLM analyzes the error and generates a response containing:
   - A diagnosis of the root cause
   - A sequence of tool calls to fix the issue (e.g., `read_file` to examine code, `write_file` to apply corrections, `create_file` to add missing headers)
   - A status indicator for whether further iterations are needed

(5) Execute the requested tool calls

(6) Repeat from step 1

This approach enables automatic resolution of common translation errors including:

- Missing header includes
- Type mismatches and implicit conversion issues
- Template instantiation errors
- Linker errors due to missing definitions

4.5 Key Design Features

4.5.1 Bottom-Up Translation with Context Propagation. The bottom-up translation order is the cornerstone of our approach, enabling context propagation that significantly improves translation quality. By translating callees before callers, we ensure that:

(1) **Consistent API usage**: When translating a caller method, the LLM sees the exact signatures and usage patterns of already-translated callees, eliminating mismatches in function calls, parameter passing, and return value handling.

(2) **Pattern reinforcement**: Common translation patterns (e.g., how `Optional<T>` is converted, how exceptions are handled) are established early in leaf methods and consistently propagated through the call graph, reducing ad-hoc decisions.

(3) **Incremental complexity**: Simple methods (often with few dependencies) are translated first, establishing project-wide conventions before tackling complex methods with deep call stacks.

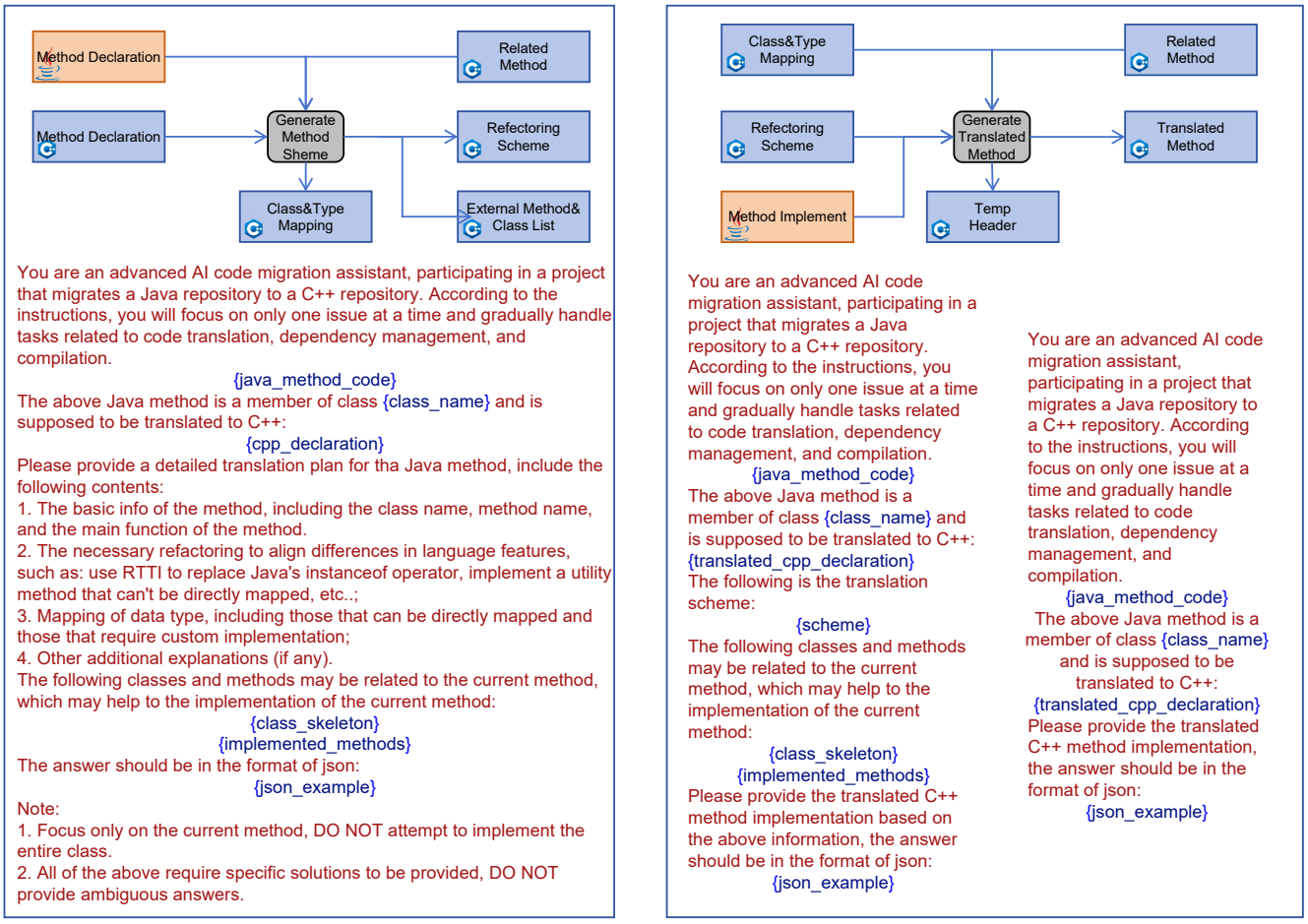

Figure 8: Method-level translation scheme and data flow. The bottom-up translation order ensures that callee methods are translated before callers, enabling context propagation where already-translated code serves as reference.

This ordering is derived automatically from the call graph using topological sort (Kahn's algorithm), which groups methods into dependency layers where all methods in layer i depend only on methods in layers 1 through $i-1$.

4.5.2 Dual-Pass Translation with Scheme Generation. Each translation unit (Header and Method) undergoes two passes:

(1) **Scheme Pass**: High-level planning without generating code. The LLM produces a structured document analyzing the translation requirements and specifying the strategy.
(2) **Code Pass**: Implementation guided by the scheme. The LLM generates actual C++ code following the strategy established in the scheme.

The two-pass approach provides several critical benefits:

- **Separation of concerns**: Strategic decisions (type mappings, refactoring needs, error handling approach) are made separately from tactical implementation, reducing cognitive load on the LLM.
- **Explicit reasoning**: The scheme serves as an explanation of the translation approach, making the process more interpretable and enabling verification of design decisions before code generation.
- **Error localization**: When translation errors occur, the scheme provides context for understanding the LLM's intent, facilitating more effective error diagnosis and correction.
- **Template for consistency**: The scheme establishes conventions that can be referenced when translating related code (e.g., all methods in a class use the same type mappings specified in the Header scheme).

4.5.3 Hierarchical Graph Representation. The bipartite graph model (Headers and Methods) provides a clean separation between structural and behavioral aspects of the codebase:

- **Structural isolation**: Header translation focuses exclusively on type definitions, field declarations, and API contracts, without the complexity of method implementations.
- **Behavioral independence**: Method translation operates on implementation logic separately from type

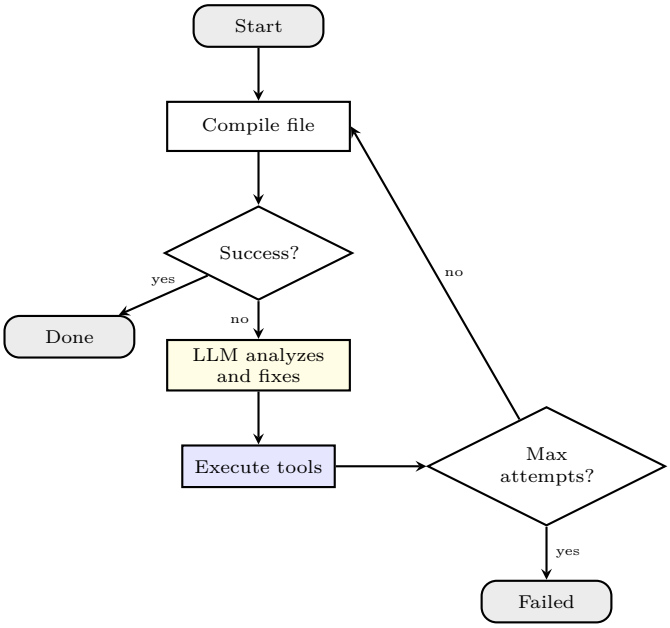

Figure 9: The compilation agent's iterative repair process.

definitions, with the translated Header providing a stable API contract.

- **Explicit dependencies**: The graph captures both structural relationships (inheritance, type usage through imports) and behavioral relationships (method calls), enabling systematic translation ordering.

This representation maps naturally to the C++ compilation model (header files and implementation files), while accommodating Java's compilation unit structure (where multiple classes may reside in one source file).

4.5.4 Heterogeneous Translation with Structural Adaptation. Unlike traditional code translation approaches that aim for syntactic one-to-one mapping, the translated codebase may exhibit structural differences from the source while preserving functional equivalence. This design philosophy is enabled by two key features:

- **Scheme-driven refactoring**: During scheme generation, the LLM can propose structural transformations that better align with target language idioms. These refactoring decisions are made before code generation, allowing systematic reorganization of the codebase structure.
- **Header-level flexibility**: The Header translation phase operates at the type-system level, where the LLM has broad freedom to redesign APIs, reorganize inheritance hierarchies, or adopt entirely different architectural patterns (e.g., replacing inheritance with templates, converting classes to namespaces).

This approach is particularly advantageous for cross-paradigm translation (e.g., Java to C++), where mechanical preservation of source structure often results in non-idiomatic code. For example:

- Java's single-inheritance class hierarchy may be restructured using C++ multiple inheritance or CRTP (Curiously Recurring Template Pattern)
- Runtime polymorphism based on interfaces may be partially replaced with compile-time polymorphism using templates
- Reflection-based designs may be restructured to use compile-time type traits and constexpr metaprogramming
- Exception-heavy error handling may be converted to RAII-based resource management with optional return values

By granting the LLM freedom to adapt project structure to target language conventions, heterogeneous translation produces more maintainable and idiomatic code than rigid structure-preserving approaches.

5 EVALUATION

This section presents a comprehensive evaluation of TOOL. We first articulate our research questions, then describe the experimental setup, datasets, and evaluation metrics. We then present both quantitative results and qualitative analysis through case studies.

5.1 Research Questions

Our evaluation is guided by the following research questions:

- **RQ1**: To what extent can TOOL successfully translate Java projects to compilable C++ code?
- **RQ2**: What is the individual contribution of each key component in TOOL?
- **RQ3**: What types of translation errors persist after the automated repair process?

5.2 Experimental Setup

5.2.1 Implementation Configuration.

Translation Framework. We implemented TOOL using Python 3.10, with the following key configurations:

- **LLM Backend**: We use `Qwen-2.5 72B, DeepSeek-V3, GPT-4o` as the LLM backend. All LLM calls use structured outputs with JSON Schema validation to ensure reliable parsing of translation schemes and generated code.
- **Static Analysis**: Java source code is parsed using a custom Java parser tool integrated with `java-callgraph2` for call graph extraction. The parser outputs project structure and method call relationships in JSON format.
- **Compilation Agent**: The agent uses `clang++` (version 20.1.8) as the C++ compiler with C++17 standard. Maximum repair attempts are set to 10 per source file.

5.2.2 Baselines. To evaluate the effectiveness of TOOL, we compare against the following baselines:

- **Direct Translation (DT)**: Files are translated sequentially without call graph analysis. Each `.java` file is translated independently to a `.cpp` file using the same LLM, with no cross-file context or scheme generation.
- **Top-Down Translation (TDT)**: Headers are translated first (similar to TOOL), but methods are translated in top-down order (caller before callee) without the bottom-up dependency ordering.
- **No-Scheme Translation (NST)**: Uses the same call graph and bottom-up ordering as TOOL, but skips the scheme generation pass, directly generating code from source.

All baselines use the same LLM model and compilation agent configuration for fair comparison.

5.3 Dataset

5.3.1 Project Selection. We selected open-source Java projects from GitHub repositories. Projects span multiple domains including concurrent programming, data parsing, testing frameworks and web frameworks. This diversity tests the framework's ability to handle different programming patterns and libraries.

Since a complete real-world project is too large to translate directly, we first extract a functional file and then find all files related to it to obtain different translation modules. After the translation is completed, we use the number of files in each translation module that can be successfully compiled as a measure of the translation's effectiveness. **Language Feature Coverage**: Projects collectively exercise various Java features that present translation challenges:

- Generics and wildcard types
- Inheritance hierarchies (both single inheritance and interface implementation)
- Exception handling (checked exceptions, try-catch-finally blocks)
- Anonymous classes and lambda expressions
- Standard library usage (Collections Framework, I/O streams, concurrency utilities)

5.3.2 Dataset Statistics. Table 2 summarizes key statistics of the dataset.

5.4 Evaluation Metrics

5.4.1 Primary Metric: Compilation Success Rate. The primary metric for evaluating translation effectiveness is the **compilation success rate**, defined as:

$$\text{Success Rate} = \frac{\text{Number of successfully compiled C++ files}}{\text{Total number of generated C++ files}} \times 100\% \tag{1}$$

5.5 Quantitative Results (RQ1)

5.5.1 Overall Translation Performance. Table 3 presents the overall translation performance of TOOL across all projects.

Table 2: Dataset Statistics

Translation Module	Main Class	Source Project	Category	Java Files	Methods
Module1	BulkheadBuilder	failsafe	fault tolerance	70	97
Module2	CircuitBreakerExecutor	failsafe	fault tolerance	64	84
Module3	DelayablePolicy	failsafe	fault tolerance	58	86
Module4	ExecutionImpl	failsafe	fault tolerance	54	105
Module5	FailsafeExecutor	failsafe	fault tolerance	68	131
Module6	FailurePolicy	failsafe	fault tolerance	60	94
Module7	RateLimiterBuilder	failsafe	fault tolerance	68	102
Module8	RateLimiterExecutor	failsafe	fault tolerance	61	93
Module9	RetryPolicyConfig	failsafe	fault tolerance	56	97
Module10	SyncExecutionImpl	failsafe	fault tolerance	59	97
Module11	TimeoutExecutor	failsafe	fault tolerance	55	82
Module12	UndeclaredThrowableStrategy	failsafe	fault tolerance	31	56
Module13	AdviceListenerTestCase	jvm-sandbox	AOP framework	65	152
Module14	ClassStructureByChildClassTestCase	jvm-sandbox	AOP framework	44	76
Module15	DebugLogExceptionModule	jvm-sandbox	AOP framework	31	85
Module16	EventWatchBuilderTestCase	jvm-sandbox	AOP framework	38	122
Module17	OrGroupFilter	jvm-sandbox	AOP framework	27	86
Module18	ProgressPrinter	jvm-sandbox	AOP framework	14	11
Module19	TestIssues217	jvm-sandbox	AOP framework	55	129
Module20	TestIssues256	jvm-sandbox	AOP framework	40	118
Module21	ReadRowHolder	easyexcel	Excel processing	10	13
Module22	ReadSheetHolder	easyexcel	Excel processing	28	49
Module23	Cookie	httpclient	HTTP client	20	79
Module24	JSONParser	hutool	utility library	17	17
Module25	PerMessageDeflateExtensionTest	Java-WebSocket	WebSocket	39	104
Module26	Draft_6455Test	Java-WebSocket	WebSocket	38	87
Total				**1293**	**2440**

Table 3: Overall Translation Performance Summary

compile unit	file count	cpp file count	method count	success count	success rate
AdviceListenerTestCase	65	18	205	14	77.78%
BulkheadBuilder	70	15	140	10	66.67%
CircuitBreakerExecutor	64	13	141	12	92.31%
ClassStructureByChildClassTestCase	44	10	89	9	90.00%
Cookie	20	7	93	4	57.14%
DebugLogExceptionModule	31	9	138	8	88.89%
DelayablePolicy	58	13	123	10	76.92%
Draft_6455Test	38	13	109	10	76.92%
EventWatchBuilderTestCase	38	11	174	10	90.91%
ExecutionImpl	54	12	149	10	83.33%
FailsafeExecutor	68	14	170	9	64.29%
FailurePolicy	60	13	132	11	84.62%
Issue231Test	61	15	139	13	86.67%
Issue260Test	62	14	123	12	85.71%
JSONParser	17	4	20	4	100.00%
OrGroupFilter	27	8	138	7	87.50%
PerMessageDeflateExtensionTest	39	14	123	9	64.29%
ProgressPrinter	14	2	37	2	100.00%
RateLimiterBuilder	68	15	155	12	80.00%
RateLimiterExecutor	61	14	146	7	50.00%
ReadRowHolder	10	2	16	2	100.00%
ReadSheetHolder	28	5	51	4	80.00%
RetryPolicyConfig	56	12	133	9	75.00%
SyncExecutionImpl	59	12	142	9	75.00%
TestIssues217	55	13	193	10	76.92%
TestIssues256	40	12	170	10	83.33%
TimeoutExecutor	55	12	116	10	83.33%
UndeclaredThrowableStrategy	31	6	62	4	66.67%
TOTAL ALL PROJECTS	**1293**	**308**	**3427**	**241**	**78.24%**

REFERENCES

[1] Xinyun Chen, Chang Liu, and Dawn Song. 2018. Tree-to-tree neural networks for program translation. *Advances in neural information processing systems* 31 (2018).

[2] Yuyan Chen, Qiang Fu, Yichen Yuan, Zhihao Wen, Ge Fan, Dayiheng Liu, Dongmei Zhang, Zhixu Li, and Yanghua Xiao. 2023. Hallucination detection: Robustly discerning reliable answers in large language models. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 245–255.

[3] Mehdi Drissi, Olivia Watkins, Aditya Khant, Vivaswat Ojha, Pedro Sandoval, Rakia Segev, Eric Weiner, and Robert Keller. 2018. Program language translation using a grammar-driven tree-to-tree model. *arXiv preprint arXiv:1807.01784* (2018).

[4] Jaemin Hong and Sukyoung Ryu. 2025. Type-migrating C-to-Rust translation using a large language model. *Empirical Software Engineering* 30, 1 (2025), 3.

[5] Ali Reza Ibrahimzada, Kaiyao Ke, Mrigank Pawagi, Muhammad Salman Abid, Rangeet Pan, Saurabh Sinha, and Reyhaneh Jabbarvand. 2025. AlphaTrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2454–2476.

[6] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards mitigating LLM hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 1827–1843.

[7] Mingsheng Jiao, Tingrui Yu, Xuan Li, Guanjie Qiu, Xiaodong Gu, and Beijun Shen. 2023. On the evaluation of neural code translation: Taxonomy and benchmark. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1529–1541.

[8] Fang Liu, Yang Liu, Lin Shi, Zhen Yang, Li Zhang, Xiaoli Lian, Zhongqi Li, and Yuchi Ma. 2026. Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code. *arXiv preprint arXiv:2404.00971v3* (2026). arXiv:2404.00971 [cs.SE]

[9] Jiaqi Liu, Fengming Zhang, Xin Zhang, Zhiwen Yu, Liang Wang, Yao Zhang, and Bin Guo. 2024. hmcodetrans: Human–machine interactive code translation. *IEEE Transactions on Software Engineering* 50, 5 (2024), 1163–1181.

[10] Feng Luo, Kexing Ji, Cuiyun Gao, Shuzheng Gao, Jia Feng, Kui Liu, Xin Xia, and Michael R Lyu. 2025. Integrating Rules and Semantics for LLM-Based C-to-Rust Translation. *arXiv preprint arXiv:2508.06926* (2025).

[11] Aniketh Malyala, Katelyn Zhou, Baishakhi Ray, and Saikat Chakraborty. 2023. On ML-based program translation: perils and promises. In *2023 IEEE/ACM 45th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. IEEE, 60–65.

[12] Potsawee Manakul, Adian Liusie, and Mark Gales. 2023. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*. 9004–9017.

[13] Vikram Nitin, Rahul Krishna, and Baishakhi Ray. 2024. Spectra: Enhancing the code translation ability of language models by generating multi-modal specifications. *arXiv preprint arXiv:2405.18574* (2024).

[14] Y. Ou, Z. Zhang, et al. 2025. K3Trans: Evolving Triple Knowledge-Augmented LLMs for Code Translation in Repository Context. *arXiv preprint arXiv:2503.18305* (2025). https://arxiv.org/html/2503.18305v3

[15] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[16] Vipula Rawte, Amit Sheth, and Amitava Das. 2023. A survey of hallucination in large foundation models. *arXiv preprint arXiv:2309.05922* (2023).

[17] Sohini Roychowdhury. 2024. Journey of hallucination-minimized generative ai solutions for financial decision makers. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 1180–1181.

[18] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised translation of programming languages. *Advances in neural information processing systems* 33 (2020), 20601–20611.

[19] Yanli Wang, Yanlin Wang, Suiquan Wang, Daya Guo, Jiachi Chen, John Grundy, Xilin Liu, Yuchi Ma, Mingzhi Mao, Hongyu Zhang, et al. 2024. Repotransbench: A real-world benchmark for repository-level code translation. *arXiv preprint arXiv:2412.17744* (2024).

[20] Jia-Yu Yao, Kun-Peng Ning, Zhen-Hui Liu, Mu-Nan Ning, Yu-Yang Liu, and Li Yuan. 2023. Llm lies: Hallucinations are not bugs, but features as adversarial examples. *arXiv preprint arXiv:2310.01469* (2023).

[21] Zhiqiang Yuan, Wenjun Mao, Zhuo Chen, Xiyue Shang, Chong Wang, Yiling Lou, and Xin Peng. 2025. Project-Level C-to-Rust Translation via Synergistic Integration of Knowledge Graphs and Large Language Models. *arXiv preprint arXiv:2510.10956* (2025).

[22] Xing Zhang, Jiaheng Wen, Fangkai Yang, Pu Zhao, Yu Kang, Junhao Wang, Maoquan Wang, Yufan Huang, Elsie Nallipogu, Qingwei Lin, et al. 2025. Skeleton-Guided-Translation: A Benchmarking Framework for Code Repository Translation with Fine-Grained Quality Evaluation. *arXiv preprint arXiv:2501.16050* (2025).

[23] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 481–503.